МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по практической работе №1

по дисциплине

«Стили и методы программирования. Часть 2 (МДН)»

на тему

«Установка и настройка среды логического программирования Prolog»

Выполнил:
Лебедевич Артем Владимирович
магистрант 2 курса группы 556301

Преподаватель:
Парамонов Антон Иванович
кандидат технических наук
доцент

Дата: 24.09.2026

Минск, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы 2

2 Используемое ПО 3

3 Ход выполнения работы 4
3.1 Установка и настройка 4
3.2 Освоение команд top-level 6
3.3 Текст программы 10
3.4 Результаты контрольных запусков 11

4 Анализ результатов 15

5 Выводы 18

Приложение А. Файл для демонстрации диагностики 19

Приложение Б. Контрольные вопросы для Moodle 20

1 ЦЕЛЬ РАБОТЫ

Освоить установку и настройку инструментальных средств логического программирования на примере SWI-Prolog, получить навыки работы с интерактивной средой (top-level), загрузки и выполнения простейших программ.

2 ИСПОЛЬЗУЕМОЕ ПО

Параметры программной среды приведены в таблице 2.1.

Таблица 2.1 – Программная среда

Параметр	Значение
Реализация Prolog	SWI-Prolog 10.0.2 (threaded, 64 bits), сборка `arm64-darwin`
Способ установки	менеджер пакетов Homebrew, формула `swi-prolog` (лицензия BSD-2-Clause)
Операционная система	macOS 26.6.2 (сборка 25G83)
Аппаратная платформа	Apple M4 Pro, архитектура arm64
Интерпретатор	`/opt/homebrew/bin/swipl`, консольный top-level в терминале zsh
Редактор	Visual Studio Code (внешний редактор исходных файлов)
Просмотр исходников	PlDoc — встроенный веб-сервер документации SWI-Prolog
Рабочая директория	`~/PycharmProjects/prolog/lab_01`

Обоснование выбора. SWI-Prolog — свободная, активно развиваемая реализация, совместимая со стандартом ISO и синтаксисом Эдинбургского Пролога, в котором написаны учебные примеры; она ставится одной командой под macOS на Apple Silicon, тогда как Visual Prolog работает только под Windows. Кроме того, SWI-Prolog содержит развитые средства отладки (трассировщик, диагностику при загрузке) и встроенную справку, что важно на этапе освоения языка.

3 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

3.1 Установка и настройка

Интерпретатор установлен из репозитория Homebrew командой `brew install swi-prolog`; вместе с ним установлены зависимости (`gmp`, `libarchive`, `openssl@3`, `pcre2`, `unixodbc` и др.). Работоспособность проверена командами оболочки `which swipl` и `swipl --version` (рисунок 3.1).

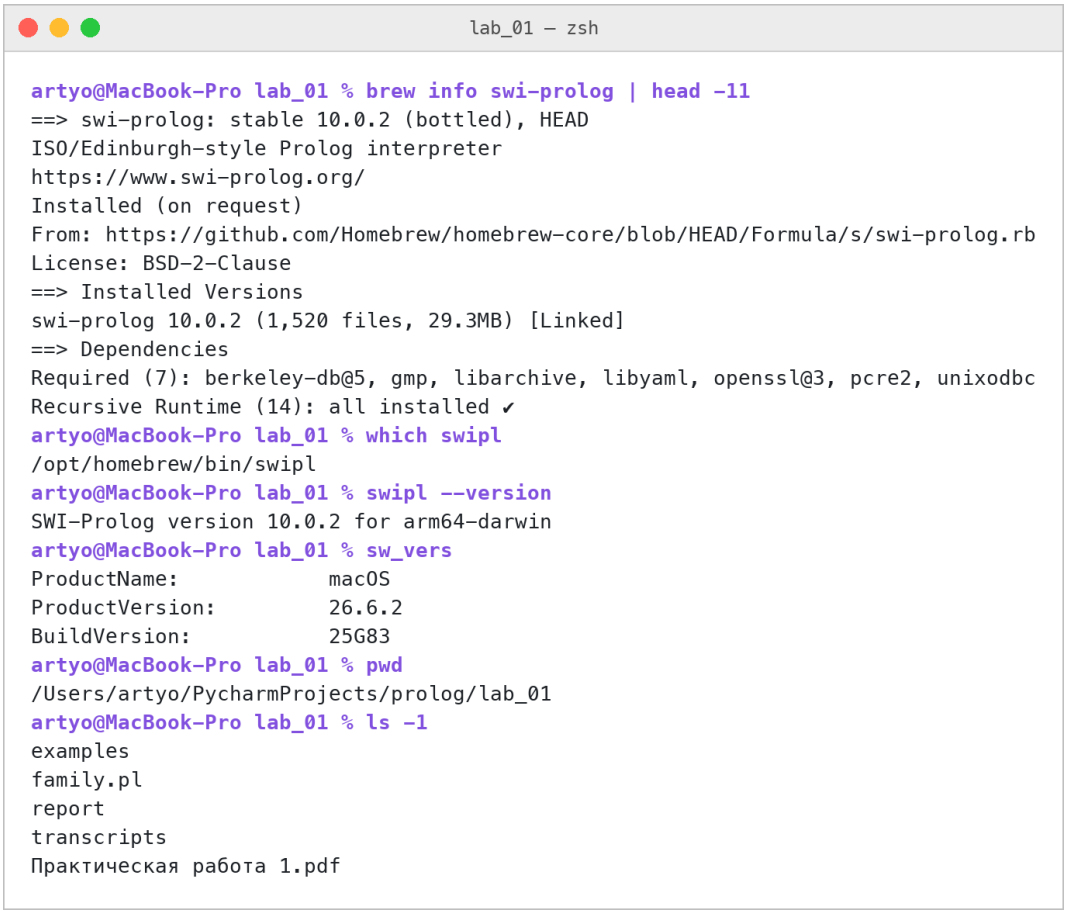


Рисунок 3.1 – Сведения об установленном пакете, версия интерпретатора, ОС и рабочая директория

Для файлов программ создана рабочая директория `lab_01` внутри репозитория `prolog`. Интерпретатор запускается из неё, поэтому она становится текущей директорией Prolog, и файлы загружаются по короткому имени без пути. Текущая директория проверяется запросом `working_directory(D, D)`, содержимое — запросом `ls`; при необходимости директорию меняют запросом `working_directory(_, 'путь')` или `cd('путь')`. Запуск интерпретатора, вывод версии и проверка рабочей директории показаны на рисунке 3.2. Запрос `version.` повторно выводит приветствие с номером версии, а флаг `version` хранит её в числовом виде: 100002 соответствует версии 10.0.2.

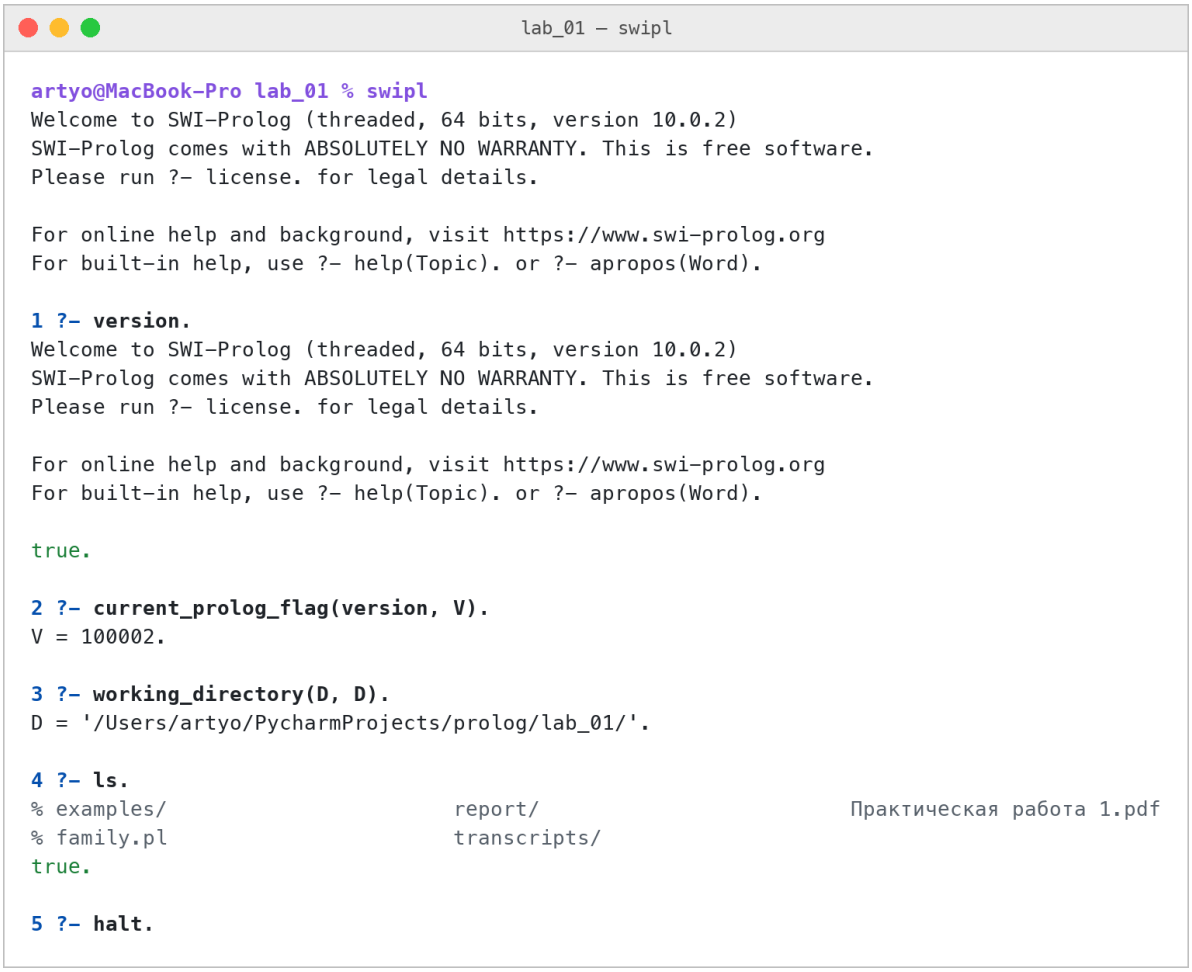


Рисунок 3.2 – Запуск интерпретатора, запрос версии и проверка рабочей директории

Исходные тексты редактируются во внешнем редакторе (VS Code), а из среды SWI-Prolog их удобно просматривать через PlDoc — встроенный веб-сервер документации. Он запускается запросом `doc_server(4040)` и показывает файлы рабочей директории (рисунок 3.3) и исходный текст с подсветкой синтаксиса: головы определяемых предикатов выделены красным, переменные — курсивом, комментарии — зелёным (рисунок 3.4). Встроенный редактор вызывается запросом `edit(family)`; какой редактор он откроет, задаёт флаг `editor`.

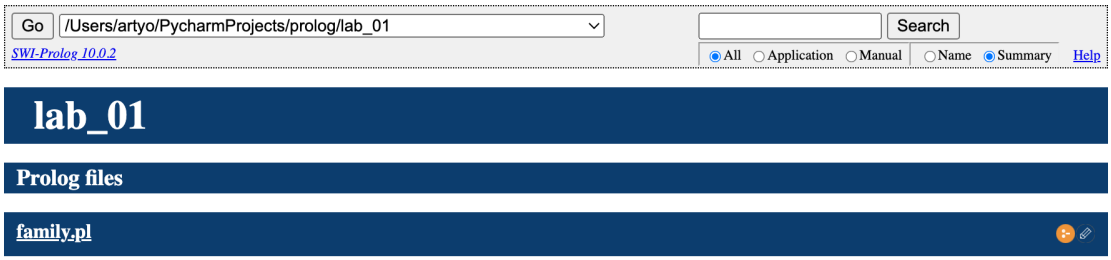


Рисунок 3.3 – Рабочая директория в веб-интерфейсе PlDoc

View source with raw comments or as raw

```
% ============================================
% Минимальная программа для проверки среды
% SWI-Prolog
% --------------------------------------------
% Практическая работа №1, «Стили и методы
% программирования. Часть 2».
% Предметная область: родственные отношения.
%
% Загрузка:  swipl family.pl
%       или  ?- [family].
% ============================================

% --- Факты: родительские отношения ---
% parent(X, Y): X является родителем Y.
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).

% --- Факты: пол ---
male(tom).
male(bob).
male(pat).
male(jim).
female(liz).
female(ann).

% --- Правила ---
% father(X, Y): X — отец Y, если X — родитель Y и X — мужчина.
father(X, Y) :- parent(X, Y), male(X).

% mother(X, Y): X — мать Y, если X — родитель Y и X — женщина.
mother(X, Y) :- parent(X, Y), female(X).

% grandparent(X, Z): X — дед/бабушка Z, если существует Y,
% такой что X — родитель Y, а Y — родитель Z.
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

% ancestor(X, Y): X — предок Y.
% Базовый случай: родитель является предком.
ancestor(X, Y) :- parent(X, Y).
% Рекурсивный случай: предок родителя является предком.
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y)
```

Рисунок 3.4 – Исходный текст программы `family.pl` в режиме просмотра PlDoc

3.2 Освоение команд top-level

Освоенные команды интерактивной среды перечислены в таблице 3.1. Каждая команда вводится после приглашения `?-` и завершается точкой.

Таблица 3.1 – Команды top-level SWI-Prolog

Команда	Назначение
`swipl family.pl`	запуск интерпретатора из оболочки с загрузкой файла
`?- [family].`	загрузка (консультация) файла `family.pl`; расширение `.pl` подставляется автоматически
`?- consult('family.pl').`	то же в явной форме; при повторной загрузке клаузы файла заменяются, а не дублируются
`?- make.`	перезагрузка всех файлов, изменённых после последней загрузки
`?- listing(ancestor/2).`	вывод клауз предиката в нормализованном виде
`?- version.`	вывод сведений о версии

Команда	Назначение
`?- help(halt/0).`	справка по предикату из встроенного руководства
`?- apropos(consult).`	поиск по руководству по ключевому слову
`?- working_directory(D, D).`	получение текущей рабочей директории
`?- ls.`	список файлов текущей директории
`?- trace.`	включение пошаговой трассировки следующего запроса
`?- notrace.`	выключение трассировки; среда остаётся в режиме отладки `[debug]`
`?- nodebug.`	выход из режима отладки
`?- halt.`	завершение работы интерпретатора (или Ctrl+D)
`;`	запрос следующего решения (возврат)
`Enter` или `.`	прекращение перебора решений

Загрузка файла из командной строки и из top-level, а также вывод предиката командой `listing` показаны на рисунке 3.5. Команды `[family].` и `consult('family.pl')` отвечают `true.`, а `listing/2` печатает клаузы `ancestor/2` с переименованными переменными и стандартными отступами.

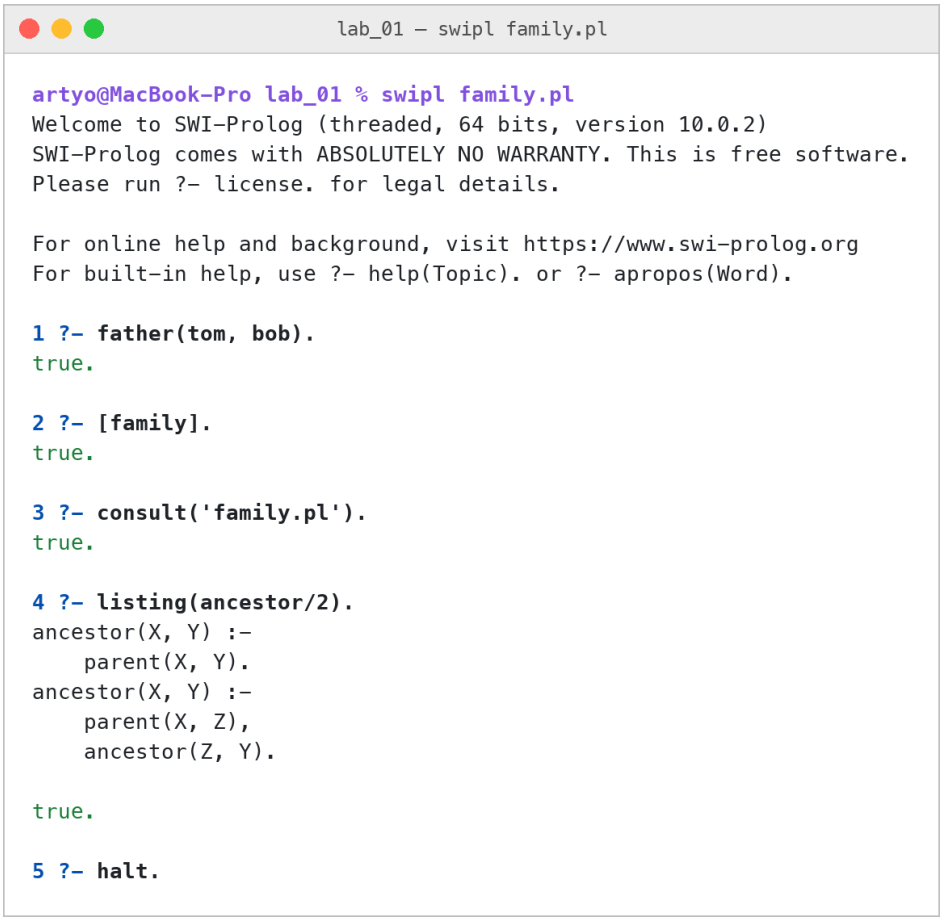


Рисунок 3.5 – Загрузка программы и просмотр клауз предиката

Встроенная справка (рисунок 3.6) выводится прямо в терминал. При первом вызове `help/1` заметна задержка в несколько секунд: система загружает индекс руководства.

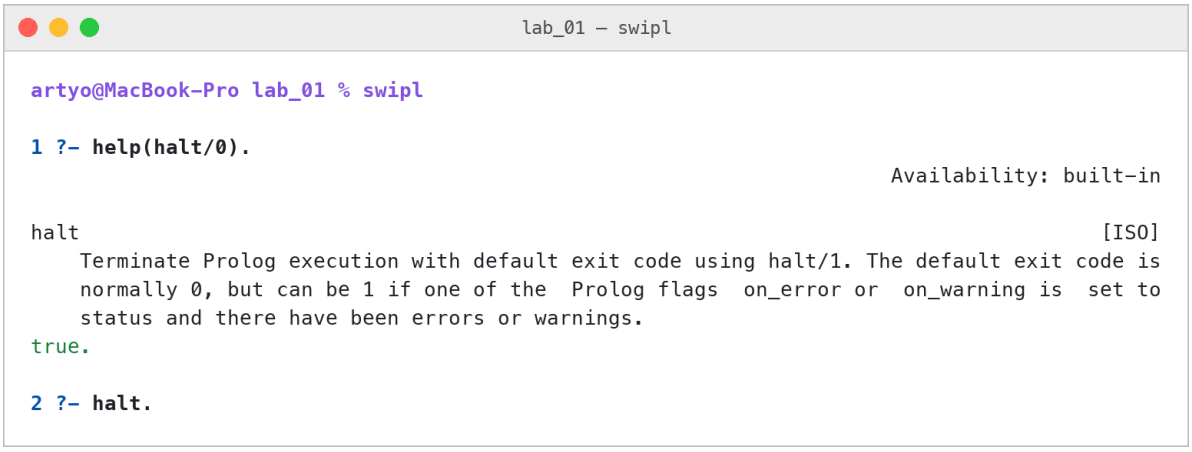


Рисунок 3.6 – Справка по предикату `halt/0`

Трассировка запроса `grandparent(X, jim)` приведена на рисунке 3.7. Трассировщик останавливается на каждом *порту* процедурной модели Бёрда: `Call` — вызов цели, `Exit` — успешный выход, `Fail` — неуспех, `Redo` — повторный вход при возврате. Нажатие `c` (или Enter) выполняет шаг (`creep`).

В скобках указана глубина вложенности цели: 12 — `grandparent`, 13 — вызовы `parent` из его тела. Хорошо видно, как Пролог перебирает факты `parent(X, Y)` сверху вниз и для каждого проверяет вторую цель `parent(Y, jim)`: пары `tom--bob`, `tom--liz`, `bob--ann` отбрасываются, и только для `bob--pat` обе цели выполняются. После нажатия `;` перебор продолжается с последней точки выбора, но других решений нет, и запрос завершается ответом `false`.



Рисунок 3.7 – Пошаговая трассировка запроса `grandparent(X, jim)`

3.3 Текст программы

Программа «Родственные отношения» из Приложения 1 к заданию записана в формате SWI-Prolog в файл `lab_01/family.pl`. База знаний содержит пять фактов `parent/2`, шесть фактов о поле (`male/1`, `female/1`) и четыре правила. Правила `father/2` и `mother/2` уточняют отношение «родитель» полом, `grandparent/2` строит цепочку из двух отношений `parent` через общую переменную `Y`, а `ancestor/2` — рекурсивное правило с базовым случаем (родитель — предок) и рекурсивным шагом (предок родителя — тоже предок).

```
% ============================================
% Минимальная программа для проверки среды
% SWI-Prolog
% --------------------------------------------
% Практическая работа №1, «Стили и методы
% программирования. Часть 2».
% Предметная область: родственные отношения.
%
% Загрузка:  swipl family.pl
%       или  ?- [family].
% ============================================

% --- Факты: родительские отношения ---
% parent(X, Y): X является родителем Y.
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).

% --- Факты: пол ---
male(tom).
male(bob).
male(pat).
male(jim).
female(liz).
female(ann).

% --- Правила ---
% father(X, Y): X — отец Y, если X — родитель Y и X — мужчина.
father(X, Y) :- parent(X, Y), male(X).

% mother(X, Y): X — мать Y, если X — родитель Y и X — женщина.
mother(X, Y) :- parent(X, Y), female(X).

% grandparent(X, Z): X — дед/бабушка Z, если существует Y,
% такой что X — родитель Y, а Y — родитель Z.
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

% ancestor(X, Y): X — предок Y.
```

```
% Базовый случай: родитель является предком.
ancestor(X, Y) :- parent(X, Y).
% Рекурсивный случай: предок родителя является предком.
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

3.4 Результаты контрольных запусков

Все 13 запросов из Приложения 2 выполнены после загрузки программы командой `swipl family.pl`. Сводка результатов приведена в таблице 3.2; результаты совпадают с ожидаемыми.

Таблица 3.2 – Результаты контрольных запросов

№	Запрос	Полученный ответ	Что демонстрирует
1	`parent(tom, bob).`	`true.`	наличие факта
2	`parent(bob, tom).`	`false.`	отсутствие факта
3	`male(tom).`	`true.`	проверка факта
4	`female(tom).`	`false.`	отрицательный факт
5	`father(tom, bob).`	`true.`	работа правила
6	`mother(liz, ann).`	`false.`	правило не выполняется
7	`parent(tom, X).`	`X = bob ; X = liz.`	перебор решений
8	`parent(X, pat).`	`X = bob.`	поиск родителя
9	`grandparent(X, jim).`	`X = bob ; false.`	правило `grandparent`
10	`ancestor(tom, X).`	`bob, liz, ann, pat, jim; false.`	рекурсивное правило
11	`parent(tom, X). + ;`	`X = bob ; X = liz.`	механизм возврата
12	`ancestor(X, jim). + ;`	`pat, tom, bob; false.`	поиск всех предков
13	`parent(X, Y). + ;`	пять пар X–Y	полный перебор базы

Проверка фактов и правил (запросы 1–6), рисунок 3.8. Запросы без переменных — это вопросы «да/нет». Запросы 1 и 3 совпадают с фактами базы и получают ответ `true`. Факта `parent(bob, tom)` в базе нет, а Пролог работает в предположении замкнутого мира: всё, что нельзя вывести, считается ложным, поэтому ответ — `false` (запросы 2 и 4). В запросе 5 правило `father` сводит цель к двум подцелям `parent(tom, bob)` и `male(tom)`, обе выполняются. В запросе 6 первая же подцель `parent(liz, ann)` неуспешна, так как мать `ann` в базе не указана.

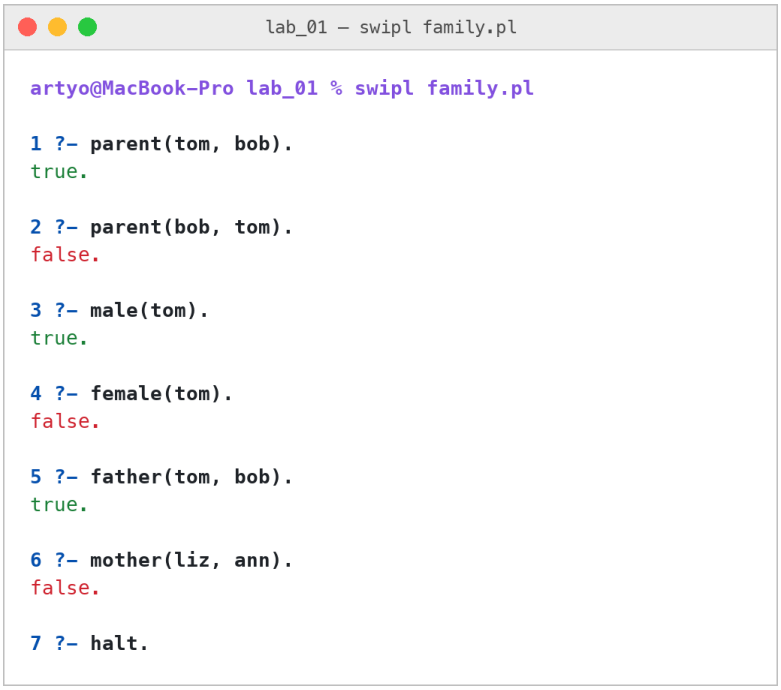


Рисунок 3.8 – Запросы 1–6: проверка фактов и правил

Запросы с переменными (7–10), рисунок 3.9. Пролог ищет такие значения переменных, при которых цель выводима, унифицируя запрос с головами клауз. Для `parent(tom, X)` подходят два факта, и после `;` выдаётся второе решение. В запросе 8 решение единственное, и SWI-Prolog сразу ставит точку: благодаря индексации по второму аргументу он знает, что других подходящих клауз нет. В запросе 9 найден дед `jim` — `bob`; после `;` система проверяет оставшиеся варианты и отвечает `false`. Запрос 10 перебирает всех потомков `tom`: сначала детей по нерекурсивной клаузе (`bob`, `liz`), затем более далёких потомков по рекурсивной (`ann`, `pat`, `jim`).

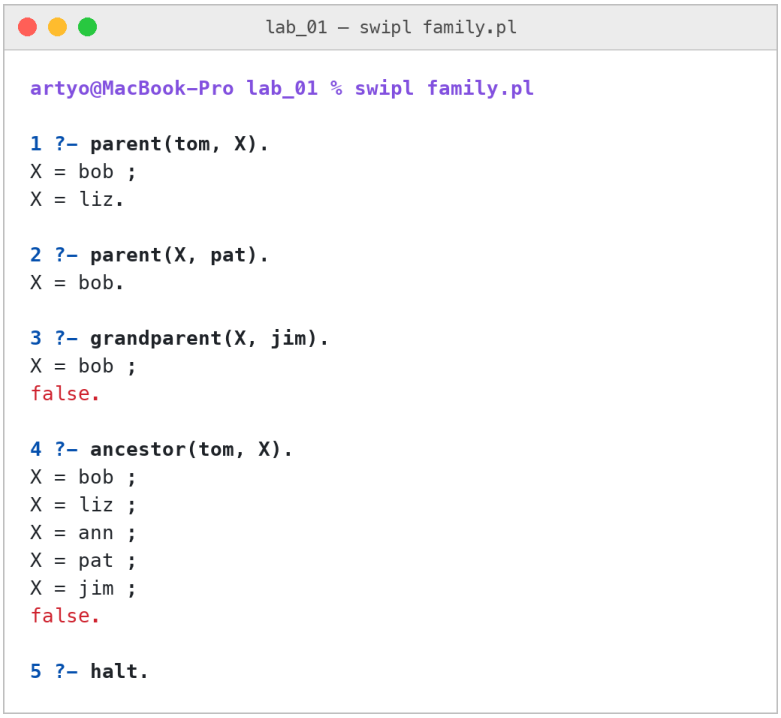


Рисунок 3.9 – Запросы 7–10: поиск решений

Возврат (запросы 11–13), рисунок 3.10. После каждого решения, если остались непроверенные альтернативы, top-level ждёт ввода: `;` заставляет систему вернуться к последней точке выбора и искать следующее решение. Порядок предков в запросе 12 (`pat`, `tom`, `bob`) определяется стратегией поиска в глубину: сначала срабатывает первая клауза `ancestor` (прямой родитель `pat`), затем рекурсивная, в которой факты `parent(X, Z)` перебираются в порядке записи. Запрос 13 последовательно выдаёт все пять пар базы. Дополнительно показан встроенный предикат `findall/3`, собирающий все решения в список без ручного нажатия `;`.

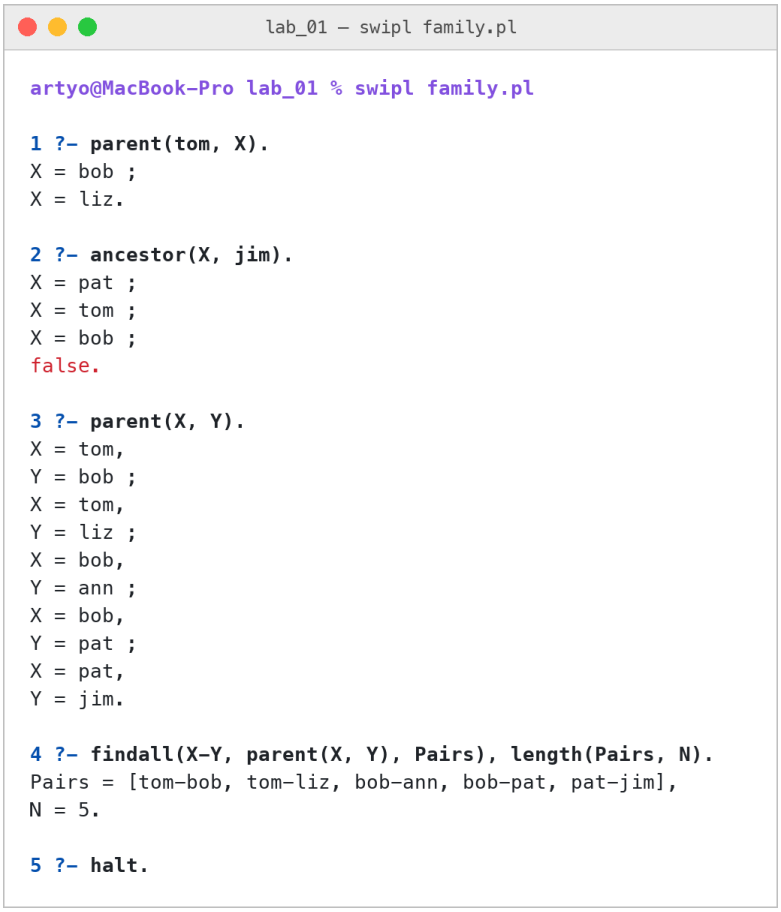


Рисунок 3.10 – Запросы 11–13: перебор решений возвратом

4 АНАЛИЗ РЕЗУЛЬТАТОВ

При работе со средой отмечены следующие особенности.

1. **Детерминированные и недетерминированные ответы.** SWI-Prolog завершает ответ точкой, если уверен, что других решений нет (`X = bob.` в запросе 8), и ждёт ввода, если осталась точка выбора (`X = bob` в запросе 9). Поэтому после последнего решения иногда появляется `false`: оно означает не ошибку, а исчерпание альтернатив. В методических указаниях для запроса 9 ожидается просто `X = bob` — это то же решение, SWI-Prolog лишь дополнительно сообщает, что больше решений нет.
2. **Представление переменных.** В трассировке несвязанные переменные выводятся как `_13014` — внутренние имена, которые меняются от запуска к запуску.
3. **Нумерация и история запросов.** Приглашение имеет вид `N ?-`, где `N` — номер запроса в истории. История сохраняется отдельно для каждой рабочей директории в `~/.config/swi-prolog/dir-history`, поэтому после перезапуска нумерация продолжается.
4. **Режимы отладки.** После `trace.` приглашение получает префикс `[trace]`, после `notrace.` — `[debug]`, и только `nodebug.` возвращает обычный режим (рисунок 3.7).

Чтобы изучить диагностику среды, был подготовлен файл `examples/errors_demo.pl` (Приложение А) с типичными ошибками. Результат его загрузки и ошибочных запросов показан на рисунках 4.1 и 4.2.

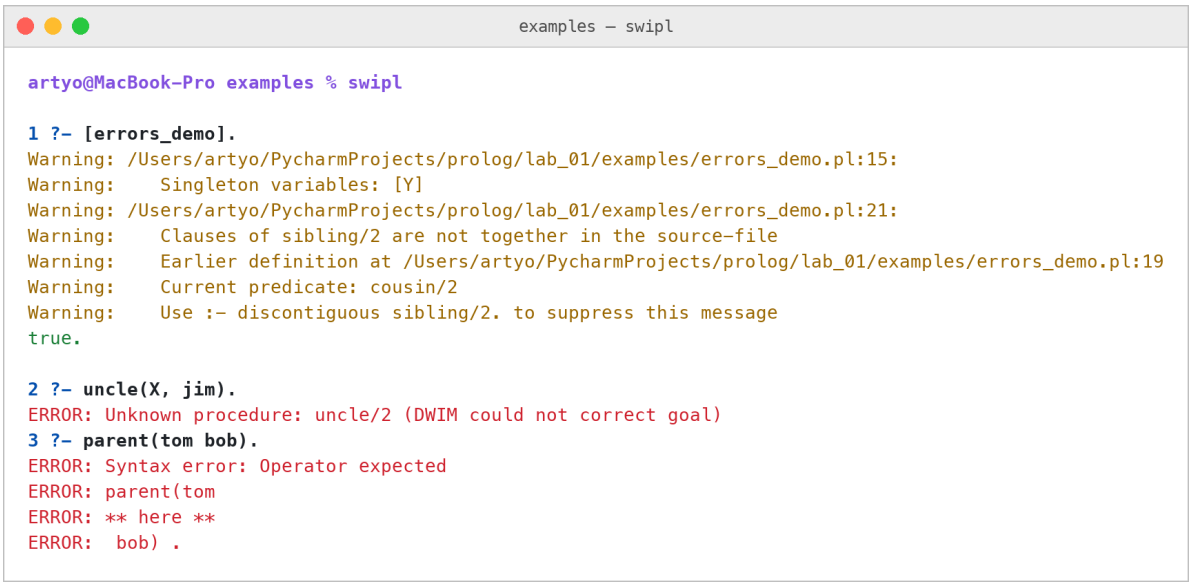


Рисунок 4.1 – Предупреждения при загрузке, неизвестный предикат и синтаксическая ошибка

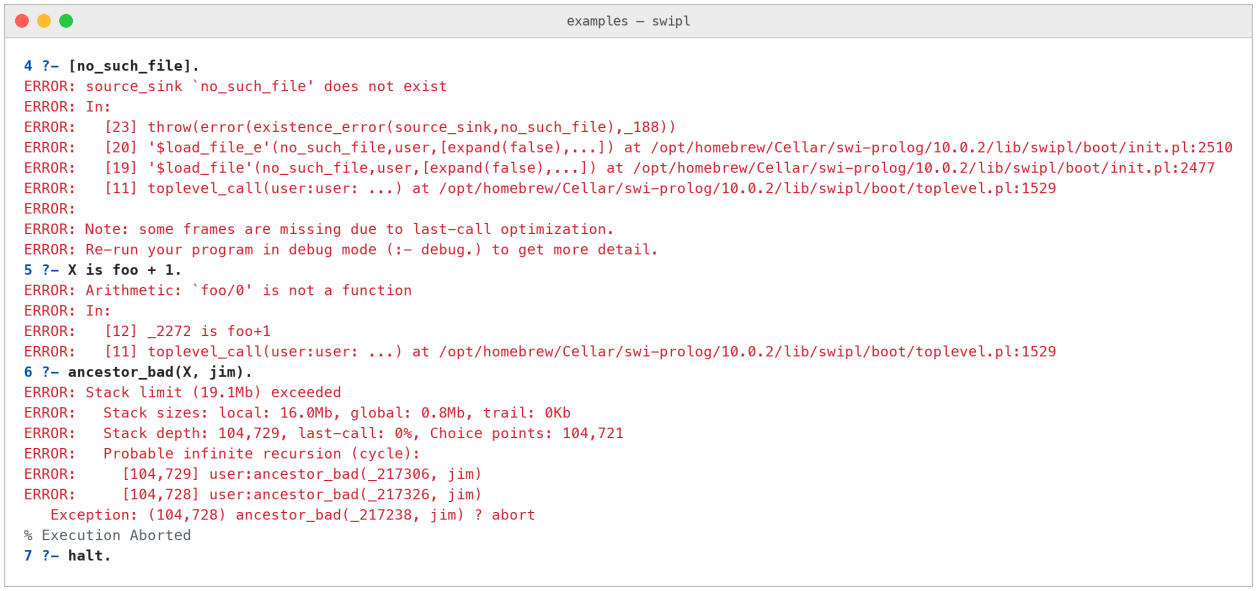


Рисунок 4.2 – Ошибки времени выполнения: отсутствующий файл, арифметика, переполнение стека

5. **Предупреждения при загрузке** не прерывают её (ответ `true`), но указывают файл и номер строки. `Singleton variables: [Y]` сообщает о переменной, которая встречается в клаузе один раз, — обычно это опечатка; `Clauses of sibling/2 are not together` сообщает, что определение предиката разорвано другим предикатом, и подсказывает директиву `discontiguous` для подавления сообщения.
6. **Неизвестный предикат.** Запрос к неопределённому `uncle/2` приводит не к `false`, а к ошибке `Unknown procedure`. Механизм DWIM («do what I mean») пытается найти похожее имя и исправить опечатку; здесь это не удалось.
7. **Синтаксическая ошибка.** При пропущенной запятой в `parent(tom bob)` выводится `Operator expected`, а место ошибки отмечается маркером `** here **`.
8. **Отсутствующий файл.** Для `[no_such_file]` возбуждается исключение `existence_error(source_sink, …)` с выводом стека вызовов.
9. **Ошибка типа в арифметике.** В выражении `X is foo + 1` атом `foo` не является арифметической функцией, о чём сообщается явно.
10. **Бесконечная рекурсия.** В левой рекурсии `ancestor_bad(X, Y) :- ancestor_bad(Z, Y), parent(X, Z)` рекурсивный вызов стоит первым и выполняется до проверки фактов, поэтому поиск уходит в бесконечный спуск. SWI-Prolog обнаруживает переполнение стека (для демонстрации предел уменьшен до 20 МБ), указывает на вероятный цикл и переходит в отладчик на порту `Exception`, из которого можно выйти командой `a` (`abort`). Корректное правило `ancestor/2` в `family.pl` этой проблемы не имеет, так как рекурсивный вызов в нём стоит после `parent(X, Z)`.

Сообщения среды информативны: они указывают тип ошибки, файл, строку и предлагают способ исправления. Главное, что нужно усвоить на этом этапе, — разница между ответом `false` (цель не выводима из базы) и ошибкой (запрос некорректен или предикат не определён).

5 ВЫВОДЫ

В ходе работы установлен и настроен SWI-Prolog 10.0.2 под macOS на Apple Silicon, подготовлена рабочая директория для программ, освоены основные команды интерактивной среды: загрузка файлов (`consult`, `[file]`, `make`), просмотр клауз (`listing`), справка (`help`, `apropos`), трассировка (`trace`, `notrace`, `nodebug`) и выход (`halt`).

Программа «Родственные отношения» загружена и проверена всеми 13 запросами из Приложения 2; полученные ответы совпадают с ожидаемыми. На практике разобраны факты, правила, унификация, рекурсия и возврат, а трассировка показала порядок, в котором Пролог перебирает клаузы и цели.

Серьёзных трудностей не возникло. Единственное расхождение с методическими указаниями — дополнительный ответ `false` после последнего решения в запросах 9, 10 и 12 — объясняется тем, что SWI-Prolog сообщает об оставшихся точках выбора. Поведение среды при ошибках изучено на специально подготовленном файле: предупреждения о singleton-переменных и разрывных определениях, ошибки неизвестного предиката, синтаксиса, отсутствующего файла, арифметики и переполнения стека при левой рекурсии.

Результат соответствует поставленной цели: среда логического программирования установлена и настроена, получены навыки работы с top-level, загрузки и выполнения простейших программ.

ПРИЛОЖЕНИЕ А. ФАЙЛ ДЛЯ ДЕМОНСТРАЦИИ ДИАГНОСТИКИ

Файл `lab_01/examples/errors_demo.pl` содержит намеренные ошибки: singleton-переменную, разрывное определение предиката и левую рекурсию.

```
% ============================================
% Файл для демонстрации диагностики SWI-Prolog:
% предупреждения при загрузке и ошибки времени
% выполнения. Используется в разделе «Анализ
% результатов» отчёта.
% ============================================

% Ограничиваем стек, чтобы переполнение наступало быстро.
:- set_prolog_flag(stack_limit, 20 000 000).

:- ensure_loaded('../family').

% 1. Singleton-переменная: Y встречается в клаузе один раз
%    (опечатка — должно было быть parent(X, _) или parent(X, Y), male(Y)).
has_child(X) :- parent(X, Y).

% 2. Разрывные (discontiguous) клаузы: sibling/2 прерывается
%    определением другого предиката.
sibling(X, Y) :- parent(P, X), parent(P, Y), X \== Y.
cousin(X, Y) :- parent(A, X), parent(B, Y), sibling(A, B).
sibling(liz, bob).

% 3. Левая рекурсия: рекурсивный вызов стоит первым,
%    поэтому поиск уходит в бесконечный спуск.
ancestor_bad(X, Y) :- ancestor_bad(Z, Y), parent(X, Z).
ancestor_bad(X, Y) :- parent(X, Y).
```

ПРИЛОЖЕНИЕ Б. КОНТРОЛЬНЫЕ ВОПРОСЫ ДЛЯ MOODLE

Ниже приведены 30 контрольных вопросов в форме, удобной для визуальной проверки. Исходный банк вопросов для загрузки в Moodle хранится в файле `moodle_questions_gift.txt` в формате GIFT.

1. Что в Прологе называется фактом?

- ☐ Клауза с головой и непустым телом, истинная при выполнении условий
- ☐ Клауза без тела, утверждение, истинное безусловно (например, parent(tom, bob).)
- ☐ Вопрос к базе знаний, вводимый после приглашения `?-`
- ☐ Директива, выполняемая при загрузке файла

Правильный ответ: Клауза без тела, утверждение, истинное безусловно (например, parent(tom, bob).).

2. Какая последовательность символов отделяет голову правила от его тела?

- ☐ :-
- ☐ ->
- ☐ :=
- ☐ <-

Правильный ответ: :-.

3. Какой запрос загружает файл family.pl в интерактивной среде SWI-Prolog?

- ☐ `?-` [family].
- ☐ `?-` load family.
- ☐ `?-` open(family).
- ☐ `?-` import(family).

Правильный ответ: `?-` [family].

4. Каким встроенным предикатом завершают работу интерпретатора SWI-Prolog?

- ☐ exit.
- ☐ quit.
- ☐ halt.
- ☐ stop.

Правильный ответ: halt.

5. Что ответит SWI-Prolog на запрос `?-` parent(bob, tom). для программы family.pl?

- ☐ true.
- ☐ false.
- ☐ ERROR: Unknown procedure
- ☐ X = bob.

Правильный ответ: false.

6. Какую клавишу нажимают в top-level SWI-Prolog, чтобы получить следующее решение запроса?

- ☐ ; (точка с запятой)
- ☐ Enter

- ☐ . (точка)
- ☐ Esc

Правильный ответ: ; (точка с запятой).

7. Как называется механизм возврата к последней точке выбора для поиска альтернативного решения?

- ☐ Унификация
- ☐ Хвостовая рекурсия
- ☐ Возврат (backtracking)
- ☐ Индексация клауз

Правильный ответ: Возврат (backtracking).

8. Что такое унификация в Прологе?

- ☐ Объединение двух файлов программы в один
- ☐ Сопоставление двух термов с подбором значений переменных, при которых термы становятся одинаковыми
- ☐ Приведение всех атомов к нижнему регистру
- ☐ Удаление повторяющихся фактов из базы

Правильный ответ: Сопоставление двух термов с подбором значений переменных, при которых термы становятся одинаковыми.

9. Какие имена в Прологе обозначают переменные?

- ☐ Имена, начинающиеся со строчной буквы
- ☐ Имена, начинающиеся с заглавной буквы или символа подчёркивания
- ☐ Имена, заключённые в одинарные кавычки
- ☐ Имена, начинающиеся с цифры

Правильный ответ: Имена, начинающиеся с заглавной буквы или символа подчёркивания.

10. Чем является tom в факте parent(tom, bob).?

- ☐ Переменной
- ☐ Строкой
- ☐ Атомом (константой)
- ☐ Именем предиката

Правильный ответ: Атомом (константой).

11. Какой запрос включает пошаговую трассировку выполнения целей?

- ☐ `?-` step.
- ☐ `?-` trace.
- ☐ `?-` debug_on.
- ☐ `?-` verbose.

Правильный ответ: `?-` trace.

12. Какие порты процедурной модели (модели Бёрда) показывает трассировщик SWI-Prolog?

- ☐ Call, Exit, Redo, Fail
- ☐ Start, Stop, Pause, Resume

- ☐ Input, Output, Error, Return
- ☐ Load, Run, Trace, Halt

Правильный ответ: Call, Exit, Redo, Fail.

13. Какую роль играет первая клауза ancestor(X, Y) :- parent(X, Y). в определении ancestor/2?

- ☐ Задаёт рекурсивный шаг
- ☐ Задаёт базовый случай рекурсии: родитель является предком
- ☐ Запрещает циклы в родословной
- ☐ Объявляет типы аргументов

Правильный ответ: Задаёт базовый случай рекурсии: родитель является предком.

14. Какое первое решение вернёт запрос `?-` grandparent(X, jim).?

- ☐ X = tom
- ☐ X = bob
- ☐ X = pat
- ☐ false.

Правильный ответ: X = bob.

15. Сколько решений имеет запрос `?-` parent(X, Y). для программы family.pl?

- ☐ 3
- ☐ 4
- ☐ 5
- ☐ 6

Правильный ответ: 5.

16. Что означает ответ false. после нескольких нажатий ; в top-level?

- ☐ В программе есть ошибка
- ☐ Найденные ранее решения неверны
- ☐ Других решений нет, перебор исчерпан
- ☐ Запрос содержит синтаксическую ошибку

Правильный ответ: Других решений нет, перебор исчерпан.

17. Что делает запрос `?-` listing(ancestor/2).?

- ☐ Удаляет предикат ancestor/2 из базы
- ☐ Выводит на экран все клаузы предиката ancestor/2
- ☐ Показывает количество решений ancestor/2
- ☐ Включает трассировку ancestor/2

Правильный ответ: Выводит на экран все клаузы предиката ancestor/2.

18. Для чего служит запрос `?-` make.?

- ☐ Компилирует программу в исполняемый файл
- ☐ Перезагружает исходные файлы, изменённые после загрузки
- ☐ Создаёт новый пустой файл программы
- ☐ Собирает проект с помощью утилиты make

Правильный ответ: Перезагружает исходные файлы, изменённые после загрузки.

19. Какое сообщение выдаст SWI-Prolog на запрос к неопределённому предикату `?- uncle(X, jim).`?

- ☐ false.
- ☐ Warning: Singleton variables
- ☐ ERROR: Unknown procedure: uncle/2
- ☐ ERROR: Syntax error: Operator expected

Правильный ответ: ERROR: Unknown procedure: uncle/2.

20. О чём предупреждает сообщение Singleton variables: [Y] при загрузке файла?

- ☐ Переменная Y не может быть унифицирована
- ☐ Переменная Y встречается в клаузе только один раз, что часто означает опечатку
- ☐ Переменная Y объявлена глобальной
- ☐ Переменная Y используется в нескольких файлах

Правильный ответ: Переменная Y встречается в клаузе только один раз, что часто означает опечатку.

21. Почему правило `a(X, Y) :- a(Z, Y), parent(X, Z).` приводит к переполнению стека?

- ☐ В теле правила слишком много переменных
- ☐ Это левая рекурсия: рекурсивный вызов стоит первым и выполняется до проверки фактов, поиск уходит в бесконечный спуск
- ☐ Предикат parent/2 не определён
- ☐ SWI-Prolog не поддерживает рекурсию

Правильный ответ: Это левая рекурсия: рекурсивный вызов стоит первым и выполняется до проверки фактов, поиск уходит в бесконечный спуск.

22. В каком порядке SWI-Prolog перебирает клаузы предиката и цели в теле правила?

- ☐ Клаузы сверху вниз в порядке записи в файле, цели тела слева направо
- ☐ Клаузы в случайном порядке, цели справа налево
- ☐ Сначала правила, затем факты
- ☐ Клаузы в алфавитном порядке

Правильный ответ: Клаузы сверху вниз в порядке записи в файле, цели тела слева направо.

23. Каким запросом можно узнать текущую рабочую директорию в SWI-Prolog?

- ☐ `?- current_dir(D).`
- ☐ `?- working_directory(D, D).`
- ☐ `?- home(D).`
- ☐ `?- path(D).`

Правильный ответ: `?- working_directory(D, D).`

24. Что вернёт запрос `?- findall(X, parent(tom, X), L).`?

- ☐ L = bob
- ☐ L = [bob, liz]
- ☐ L = [tom]
- ☐ false.

Правильный ответ: L = [bob, liz].

25. Каким запросом открыть встроенную справку по предикату halt/0?

- ☐ `?- man(halt).`
- ☐ `?- info halt.`
- ☐ `?- help(halt/0).`
- ☐ `?- doc halt.`

Правильный ответ: `?- help(halt/0).`

26. Для чего служит запрос `?- apropos(consult).`?

- ☐ Загружает файл consult.pl
- ☐ Ищет в документации предикаты и разделы по ключевому слову
- ☐ Выводит исходный код consult/1
- ☐ Отключает предикат consult/1

Правильный ответ: Ищет в документации предикаты и разделы по ключевому слову.

27. Какую логическую связку обозначает запятая в теле правила?

- ☐ Конъюнкцию (логическое И)
- ☐ Дизъюнкцию (логическое ИЛИ)
- ☐ Отрицание
- ☐ Импликацию

Правильный ответ: Конъюнкцию (логическое И).

28. Какое расширение обычно имеют исходные файлы SWI-Prolog?

- ☐ .swi
- ☐ .pl
- ☐ .prg
- ☐ .lsp

Правильный ответ: .pl.

29. Как читается декларативно правило father(X, Y) :- parent(X, Y), male(X).?

- ☐ X — отец Y, если Y — родитель X и Y — мужчина
- ☐ X — отец Y, если X — родитель Y и X — мужчина
- ☐ X — отец Y или X — мужчина
- ☐ Каждый мужчина является отцом

Правильный ответ: X — отец Y, если X — родитель Y и X — мужчина.

30. Чем программа на Visual Prolog отличается от программы на SWI-Prolog?

- ☐ В Visual Prolog нет правил, только факты
- ☐ В Visual Prolog обязательны секции domains, predicates и clauses с объявлением типов
- ☐ Visual Prolog не поддерживает рекурсию
- ☐ В Visual Prolog запросы вводятся только в top-level

Правильный ответ: В Visual Prolog обязательны секции domains, predicates и clauses с объявлением типов.